



MÓDULO 4 · T11

Context y Temas

🌙 *Tema claro/oscuro con Context API*

Carlos Rodríguez Contreras · GitHub: `karrocon`

¿Qué aprenderás hoy?

En esta sesión construirás un sistema completo de temas claro/oscuro usando la Context API de React. Al terminar, serás capaz de gestionar el estado global del tema sin prop drilling y de detectar las preferencias del sistema operativo.

1

createContext

Crear un Context con un valor tipado y un valor por defecto seguro.

2

Provider con estado

Implementar un Provider que gestiona el modo de tema y lo expone al árbol.

3

useContext

Acceder al contexto desde cualquier componente con el hook `useTheme()`.

4

Temas light/dark/system

Construir un sistema de temas que respeta la preferencia del sistema operativo.

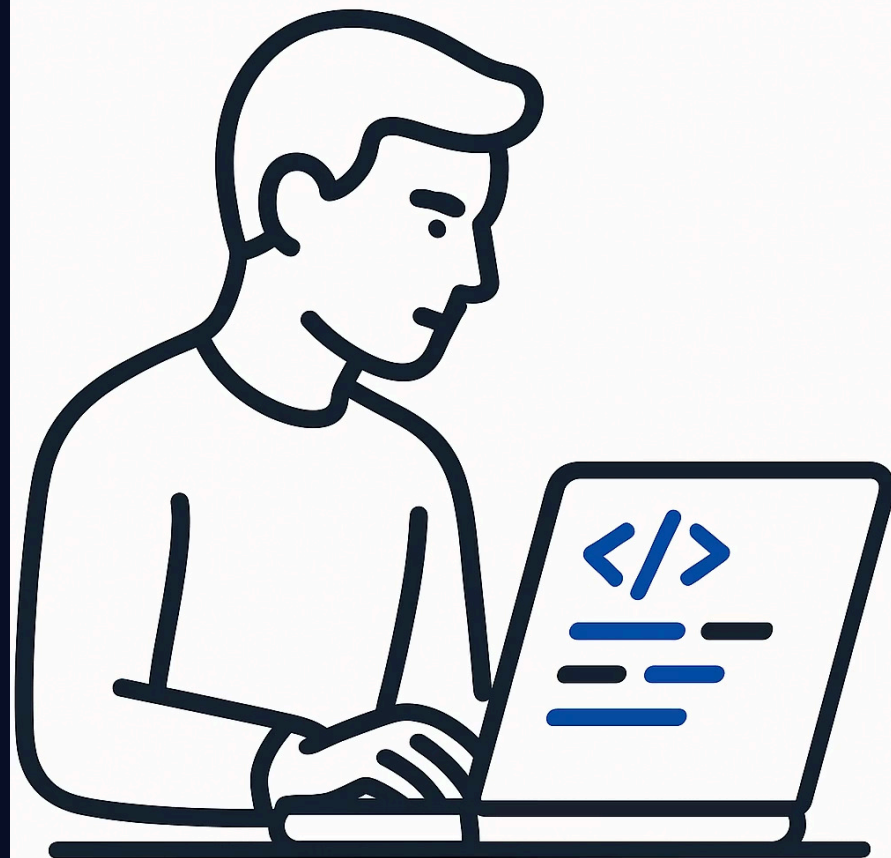
El problema sin Context

Sin Context API, pasar datos compartidos como el tema a todos los componentes obliga a perforar props por capas intermedias que no los necesitan. Esto se conoce como **prop drilling** y hace el código frágil y difícil de mantener.

// Sin Context: hay que pasar el tema por props a cada componente

```
<Screen theme={theme}>  
  <Header theme={theme} />  
  <PokemonList theme={theme}>  
    <PokemonListItem theme={theme} />  
    {/* prop drilling */}  
  </PokemonList>  
</Screen>
```

⚠ **Prop drilling:** pasar datos por capas intermedias que no los necesitan. Context resuelve este problema de raíz.



createContext + useContext

¿Cómo funciona?

La Context API sigue tres pasos: **crear** el contenedor de datos, **proveer** el valor al árbol de componentes y **consumir** el valor desde cualquier hijo, sin importar su profundidad.

- ✅ Con `useTheme()` cualquier componente accede al tema con una sola línea – sin props intermedias.

Los tres pasos

```
// 1. Crear el contexto con valor por defecto
const ThemeContext = createContext<ThemeContextValue | null>(null);

// 2. Provider — envuelve el árbol
export function ThemeProvider({ children }) {
  const [isDark, setIsDark] = useState(false);
  return (
    <ThemeContext.Provider value={{ isDark, toggle: () => setIsDark(d =>
!d)}}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Consumir desde cualquier componente
export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('Fuera de ThemeProvider');
  return ctx;
}
```

Tokens de tema

Un objeto de tema (**token de tema**) centraliza todos los colores de la UI en un único lugar tipado. Cambiar el tema implica simplemente intercambiar el objeto – todos los componentes que lo consumen se actualizan automáticamente.

```
export type Theme = {  
  background: string;  
  backgroundCard: string;  
  textPrimary: string;  
  textSecondary: string;  
  primary: string;  
};  
  
export const lightTheme: Theme = {  
  background: '#F8F9FA',  
  backgroundCard: 'FFFFFF',  
  textPrimary: '#212529',  
  textSecondary: '#868E96',  
  primary: '#CC0000',  
};
```

useColorScheme — tema del sistema

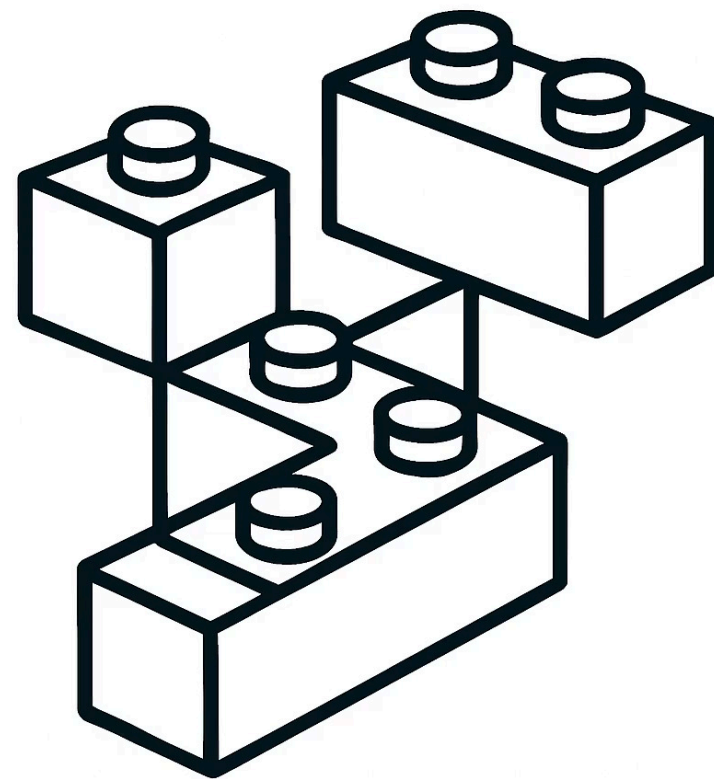
React Native expone el hook `useColorScheme` para leer la preferencia de tema del sistema operativo. Combinado con un modo manual, permite ofrecer tres opciones: **light**, **dark** y **system** (sigue al SO).

```
import { useColorScheme } from 'react-native';

export function ThemeProvider({ children }) {
  const systemScheme = useColorScheme(); // 'light' | 'dark' | null
  const [mode, setMode] = useState<'light' | 'dark' | 'system'>('system');

  const isDark = mode === 'system'
    ? systemScheme === 'dark'
    : mode === 'dark';

  const theme = isDark ? darkTheme : lightTheme;
  // ...
}
```



PokeApp — useTheme.tsx (fragmento)

La aplicación de ejemplo añade persistencia al modo elegido por el usuario mediante `AsyncStorage`. Al arrancar la app, el Provider recupera el modo guardado y lo aplica antes del primer render.

```
export function ThemeProvider({ children }: Props) {
  const systemScheme = useColorScheme();
  const [mode, setModeState] = useState<ThemeMode>('system');

  // Persistir el modo elegido por el usuario
  useEffect(() => {
    loadJSON<ThemeMode>(THEME_STORAGE_KEY).then(saved => {
      if (saved) setModeState(saved);
    });
  }, []);

  const setMode = useCallback((m: ThemeMode) => {
    setModeState(m);
    saveJSON(THEME_STORAGE_KEY, m); // ← persistido en AsyncStorage
  }, []);
}
```

📌 **Buena práctica:** usa `useCallback` para memorizar `setMode` y evitar renders innecesarios en los componentes que consumen el contexto.

Usar el tema en un componente

Acceso al tema

Llama a `useTheme()` dentro del componente para obtener el objeto `theme`. Luego aplica sus tokens directamente en los estilos en línea que dependen del tema activo.

❗ Los estilos estáticos siguen en `StyleSheet.create` – solo los colores dinámicos van en línea.

Ejemplo: PokemonListItem

```
function PokemonListItem({ pokemon }: Props) {  
  const { theme } = useTheme();  
  
  return (  
    <View style={{backgroundColor: theme.backgroundCard}}>  
      <Text style={{color: theme.textPrimary}}>  
        {pokemon.nombre}  
      </Text>  
      <Text style={{color: theme.textSecondary}}>  
        #{pokemon.indice}  
      </Text>  
    </View>  
  );  
}
```


Tour starter/ThemeContext.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Implementa el cambio entre `'light'` y `'dark'` pero aún no expone el objeto `theme` ni el booleano `isDark` al contexto – esos son los `TODO` que deberás completar.

```
const ThemeContext = createContext<ThemeContextValue | null>(null);

export function ThemeProvider({ children }) {
  const [mode, setMode] = useState<ThemeMode>('light');
  const toggle = () => setMode(m => m === 'light' ? 'dark' : 'light');

  return (
    <ThemeContext.Provider value={{ mode, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
}

// TODO: añade `theme: Theme` e `isDark: boolean` al valor
```

⚠ No modifiques la estructura del toggle ni los nombres de las variables existentes – céntrate en los `TODO` marcados arriba.

✓ SOLUCIÓN

Solución — solucion/ThemeContext.tsx

La solución completa calcula `isDark` a partir del modo, selecciona el objeto `theme` correcto y los expone junto al resto del valor del contexto. Compara tu implementación con esta referencia.

```
const isDark = mode === 'system'
  ? systemScheme === 'dark'
  : mode === 'dark';

const theme = isDark ? darkTheme : lightTheme;

return (
  <ThemeContext.Provider value={{
    mode,
    toggle,
    isDark, // ← añadido
    theme,  // ← añadido
  }}>
    {children}
  </ThemeContext.Provider>
);
```

✓ Si los colores de la app cambian al pulsar el botón de toggle y respetan el modo del sistema, ¡lo has conseguido! 🎉

Ejercicio T11

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. Al terminar, la app cambiará de tema de forma fluida y recordará tu elección.

1 Abre el fichero de inicio

Navega a `starter/ThemeContext.tsx` en tu editor.

2 Añade los objetos de tema

Define `lightTheme` y `darkTheme` con los tokens de color necesarios.

3 Calcula `isDark` y `theme`

Deriva `isDark` del modo actual y selecciona el objeto `theme` correspondiente.

4 Expón `theme` e `isDark`

Añade `theme` e `isDark` al valor del `ThemeContext.Provider`.

5 Usa `useTheme()` en un componente

Aplica los tokens del tema en `PokemonListItem` para cambiar colores dinámicamente.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominarlos te permitirá gestionar estado global de UI de forma limpia y escalable en cualquier aplicación React Native.

Concepto	Clave	Para qué sirve
createContext	<code>createContext<T null>(null)</code>	Crea el contenedor de datos
Provider	<code><Context.Provider value={...}></code>	Proporciona el valor al árbol
useContext	<code>useContext(ThemeContext)</code>	Lee el valor desde cualquier hijo
Tokens de tema	<code>Theme = { background, textPrimary... }</code>	Objeto con todos los colores
useColorScheme	<code>useColorScheme() → 'light' 'dark'</code>	Detecta el tema del sistema
Persistencia	<code>saveJSON / loadJSON</code>	Guarda el modo en AsyncStorage

✅ ¡Enhorabuena! Ahora tienes las herramientas para construir un sistema de temas completo con Context API, detección del sistema y persistencia. 🚀